



Joins and SQL Queries

(Only Important Queries)



Top 20 SQL Queries :-

Table - EmployeeDetails

EmpId	FullName	ManagerId	DateOfJoining
121	John Snow	321	01/31/2014
321	Walter White	986	01/30/2015
421	Kuldeep Rana	876	27/11/2016

Table - EmployeeSalary

EmpId	Project	Salary
121	P1	8000
321	P2	1000
421	P1	12000

Ques.1. Write a SQL query to fetch the count of employees working in project 'P1'.

Ans. Here, we would be using aggregate function count() with the SQL where clause-

```
SELECT COUNT(*) FROM EmployeeSalary WHERE Project = 'P1';
```

Ques.2. Write a SQL query to fetch employee names having salary greater than or equal to 5000 and less than or equal 10000.

Ans. Here, we will use BETWEEN in the 'where' clause to return the empld of the employees with salary satisfying the required criteria and then use it as subquery to find the fullName of the employee from EmployeeDetails table.

```
SELECT FullName
FROM EmployeeDetails
WHERE EmpId IN
(SELECT EmpId FROM EmployeeSalary
WHERE Salary BETWEEN 5000 AND 10000);
```

Ques.3. Write a SQL query to fetch project-wise count of employees sorted by project's count in descending order.

Ans. The query has two requirements - first to fetch the project-wise count and then to sort the result by that count. For project wise count, we will be using GROUPBY clause and for sorting, we will use ORDER BY clause on the alias of the project-count.

```
SELECT Project, count(EmpId) EmpProjectCount
FROM EmployeeSalary
GROUP BY Project
ORDER BY EmpProjectCount DESC;
```

Ques.4. Write a query to fetch only the first name(string before space) from the FullName column of EmployeeDetails table.

Ans. In this question, we are required to first fetch the location of the space character in the FullName field and then extract the first name out of the FullName field. For finding the location we will use LOCATE method in MySQL and CHARINDEX in SQL SERVER and for fetching the string before space, we will use SUBSTRING OR MID method.

mySQL- Using MID

```
SELECT MID(FullName, 0, LOCATE(' ',FullName)) FROM EmployeeDetails;
```

SQL Server-Using SUBSTRING

```
SELECT SUBSTRING(FullName, 0, CHARINDEX(' ',FullName)) FROM EmployeeDetails;
```

Also, we can use LEFT which returns the left part of a string till specified number of characters.

```
SELECT LEFT(FullName, CHARINDEX(' ',FullName) - 1) FROM EmployeeDetails;
```

Ques.5. Write a query to fetch employee names and salary records. Return employee details even if the salary record is not present for the employee.

Ans. Here, we can use left join with EmployeeDetail table on the left side.

```
SELECT E.FullName, S.Salary
FROM EmployeeDetails E LEFT JOIN EmployeeSalary S
ON E.Empld = S.Empld;
```

Ques.6. Write a SQL query to fetch all the Employees who are also managers from EmployeeDetails table.

Ans. Here, we have to use Self-Join as the requirement wants us to analyze the EmployeeDetails table as two different tables, each for Employee and manager records.

```
SELECT DISTINCT E.FullName
FROM EmpDetails E
INNER JOIN EmpDetails M
ON E.EmplD = M.ManagerID;
```

Ques.7. Write a SQL query to fetch all employee records from EmployeeDetails table who have a salary record in EmployeeSalary table.

Ans. Using 'Exists'-

```
SELECT * FROM EmployeeDetails E
WHERE EXISTS
(SELECT * FROM EmployeeSalary S WHERE E.Empld = S.Empld);
```

Ques.8. Write a SQL query to fetch duplicate records from a table.

Ans. In order to find duplicate records from table we can use GROUP BY on all the fields and then use HAVING clause to return only those fields whose count is greater than 1 i.e. the rows having duplicate records.

```
SELECT Empld, Project, Salary, COUNT(*)
FROM EmployeeSalary
GROUP BY Empld, Project, Salary
HAVING COUNT(*) > 1;
```

Ques.9. Write a SQL query to remove duplicates from a table without using temporary table.

Ans. Using Group By and Having clause-

```
DELETE FROM EmployeeSalary
WHERE Empld IN (
SELECT Empld
FROM EmployeeSalary
GROUP BY Project, Salary
HAVING COUNT(*) > 1));
```

Using rowid in Oracle-

```
DELETE FROM EmployeeSalary
WHERE rowid NOT IN
(SELECT MAX(rowid) FROM EmployeeSalary GROUP BY EmpId);
```

Ques.10. Write a SQL query to fetch only odd rows from table.

Ans. This can be achieved by using Row_number in SQL server-

```
SELECT E.EmpId, E.Project, E.Salary
FROM (
    SELECT *, Row_Number() OVER(ORDER BY EmpId) AS RowNumber
    FROM EmployeeSalary
) E
WHERE E.RowNumber % 2 = 1
```

Ques.11. Write a SQL query to fetch only even rows from table.

Ans. Using the same Row_Number() and checking that the remainder when divided by 2 is 0-

```
SELECT E.EmpId, E.Project, E.Salary
FROM (
    SELECT *, Row_Number() OVER(ORDER BY EmpId) AS RowNumber
    FROM EmployeeSalary
) E
WHERE E.RowNumber % 2 = 0
```

Ques.12. Write a SQL query to create a new table with data and structure copied from another table.

Ans. Using SELECT INTO command-

```
SELECT * INTO newTable FROM EmployeeDetails;
```

Ques.13. Write a SQL query to create an empty table with same structure as some other table.

Ans. Using SELECT INTO command with False 'WHERE' condition-

```
SELECT * INTO newTable FROM EmployeeDetails WHERE 1 = 0;
```

This can also done using mySQL 'Like' command with CREATE statement-

```
CREATE TABLE newTable LIKE EmployeeDetails;
```

Ques.14. Write a SQL query to fetch common records between two tables.

Ans. Using INTERSECT-

```
SELECT * FROM EmployeeSalary
INTERSECT
SELECT * FROM ManagerSalary
```

Ques.15. Write a SQL query to fetch records that are present in one table but not in another table.

Ans. Using MINUS-

```
SELECT * FROM EmployeeSalary
MINUS
SELECT * FROM ManagerSalary
```

Ques.16. Write a SQL query to find current date-time.

Ans. mySQL-

```
SELECT NOW();
```

SQL Server-

```
SELECT getdate();
```

Oracle-

```
SELECT SYSDATE FROM DUAL;
```

Ques.17. Write a SQL query to fetch all the Employees details from EmployeeDetails table who joined in Year 2016.

Ans. Using BETWEEN for the date range '01-01-2016' AND '31-12-2016'-

```
SELECT * FROM EmployeeDetails
WHERE DateOfJoining BETWEEN '01-01-2016' AND date '31-12-2016';
```

Also, we can extract year part from the joining date (using YEAR in mySQL)-

```
SELECT * FROM EmployeeDetails
WHERE YEAR(DateOfJoining) = '2016';
```

Ques.18. Write a SQL query to fetch top n records?

Ans. In mySQL using LIMIT-

```
SELECT * FROM EmployeeSalary ORDER BY Salary DESC LIMIT N
```

In SQL server using TOP command-

```
SELECT TOP N * FROM EmployeeSalary ORDER BY Salary DESC
```

In Oracle using ROWNUM-

```
SELECT * FROM (SELECT * FROM EmployeeSalary ORDER BY Salary DESC)
WHERE ROWNUM <= 3;
```

Ques.19. Write SQL query to find the nth highest salary from table.

Ans. Using Top keyword (SQL Server)-

```
SELECT TOP 1 Salary
FROM (
    SELECT DISTINCT TOP N Salary
    FROM Employee
    ORDER BY Salary DESC
)
ORDER BY Salary ASC
```

Using limit clause(mySQL)-

```
SELECT Salary FROM Employee ORDER BY Salary DESC LIMIT N-1,1;
```

Ques.20. Write SQL query to find the 3rd highest salary from table without using TOP/limit keyword.

Ans. The below SQL query make use of correlated subquery wherein in order to find the 3rd highest salary the inner query will return the count of till we find that there are two rows that salary greater than other distinct salaries.

```
SELECT Salary
FROM EmployeeSalary Emp1
WHERE 2 = (
    SELECT COUNT( DISTINCT ( Emp2.Salary ) )
    FROM EmployeeSalary Emp2
    WHERE Emp2.Salary > Emp1.Salary
)
```

For nth highest salary-

```
SELECT Salary
FROM EmployeeSalary Emp1
WHERE N-1 = (
    SELECT COUNT( DISTINCT ( Emp2.Salary ) )
    FROM EmployeeSalary Emp2
    WHERE Emp2.Salary > Emp1.Salary
)
```

Top 50 SQL Queries :-

Sample Table – Worker

WORKER_ID	FIRST_NAME	LAST_NAME	SALARY	JOINING_DATE	DEPARTMENT
001	Monika	Arora	100000	2014-02-20 09:00:00	HR
002	Niharika	Verma	80000	2014-06-11 09:00:00	Admin
003	Vishal	Singhal	300000	2014-02-20 09:00:00	HR
004	Amitabh	Singh	500000	2014-02-20 09:00:00	Admin
005	Vivek	Bhati	500000	2014-06-11 09:00:00	Admin
006	Vipul	Diwan	200000	2014-06-11 09:00:00	Account
007	Satish	Kumar	75000	2014-01-20 09:00:00	Account
008	Geetika	Chauhan	90000	2014-04-11 09:00:00	Admin

Sample Table – Bonus

WORKER_REF_ID	BONUS_DATE	BONUS_AMOUNT
1	2016-02-20 00:00:00	5000
2	2016-06-11 00:00:00	3000
3	2016-02-20 00:00:00	4000
1	2016-02-20 00:00:00	4500
2	2016-06-11 00:00:00	3500

Sample Table – Title

WORKER_REF_ID	WORKER_TITLE	AFFECTED_FROM
1	Manager	2016-02-20 00:00:00
2	Executive	2016-06-11 00:00:00
8	Executive	2016-06-11 00:00:00
5	Manager	2016-06-11 00:00:00
4	Asst. Manager	2016-06-11 00:00:00
7	Executive	2016-06-11 00:00:00
6	Lead	2016-06-11 00:00:00
3	Lead	2016-06-11 00:00:00

To prepare the sample data, you can run the following queries in your database query executor or on the SQL command line. We've tested them with MySQL Server 5.7 and MySQL Workbench 6.3.8 query browser. You can also download these Softwares and install them to carry on the SQL exercise.

SQL Script To Seed Sample Data:

```
CREATE DATABASE ORG;
```

```
SHOW DATABASES;
```

```
USE ORG;
```

```
CREATE TABLE Worker (
```

```
    WORKER_ID INT NOT NULL PRIMARY KEY AUTO_INCREMENT,
```

```
    FIRST_NAME CHAR(25),
```

```
    LAST_NAME CHAR(25),
```

```
    SALARY INT(15),
```

```
    JOINING_DATE DATETIME,
```

```
    DEPARTMENT CHAR(25)
```

```
);
```

```
INSERT INTO Worker
```

```
    (WORKER_ID, FIRST_NAME, LAST_NAME, SALARY, JOINING_DATE, DEPARTMENT)
```

```
VALUES
```

```
    (001, 'Monika', 'Arora', 100000, '14-02-20 09.00.00', 'HR'),
```

```
    (002, 'Niharika', 'Verma', 80000, '14-06-11 09.00.00', 'Admin'),
```

```
    (003, 'Vishal', 'Singhal', 300000, '14-02-20 09.00.00', 'HR'),
```

```
    (004, 'Amitabh', 'Singh', 500000, '14-02-20 09.00.00', 'Admin'),
```

```
    (005, 'Vivek', 'Bhati', 500000, '14-06-11 09.00.00', 'Admin'),
```

```
    (006, 'Vipul', 'Diwan', 200000, '14-06-11 09.00.00', 'Account'),
```

```
    (007, 'Satish', 'Kumar', 75000, '14-01-20 09.00.00', 'Account'),
```

```
    (008, 'Geetika', 'Chauhan', 90000, '14-04-11 09.00.00', 'Admin');
```

```
CREATE TABLE Bonus (
```

```
    WORKER_REF_ID INT,
```

```
    BONUS_AMOUNT INT(10),
```

```
    BONUS_DATE DATETIME,
```

```
    FOREIGN KEY (WORKER_REF_ID)
```

```
        REFERENCES Worker(WORKER_ID)
```

```
    ON DELETE CASCADE
```

```
);
```

```
INSERT INTO Bonus
```

```
    (WORKER_REF_ID, BONUS_AMOUNT, BONUS_DATE) VALUES
```

```
    (001, 5000, '16-02-20'),
```

```
    (002, 3000, '16-06-11'),
```

```
    (003, 4000, '16-02-20'),
```

```
    (001, 4500, '16-02-20'),
```

```
    (002, 3500, '16-06-11');
```

```
CREATE TABLE Title (
```

```
    WORKER_REF_ID INT,
```

```

        WORKER_TITLE CHAR(25),
        AFFECTED_FROM DATETIME,
        FOREIGN KEY (WORKER_REF_ID)
            REFERENCES Worker(WORKER_ID)
        ON DELETE CASCADE
    );

INSERT INTO Title
    (WORKER_REF_ID, WORKER_TITLE, AFFECTED_FROM) VALUES
    (001, 'Manager', '2016-02-20 00:00:00'),
    (002, 'Executive', '2016-06-11 00:00:00'),
    (008, 'Executive', '2016-06-11 00:00:00'),
    (005, 'Manager', '2016-06-11 00:00:00'),
    (004, 'Asst. Manager', '2016-06-11 00:00:00'),
    (007, 'Executive', '2016-06-11 00:00:00'),
    (006, 'Lead', '2016-06-11 00:00:00'),
    (003, 'Lead', '2016-06-11 00:00:00');

```

Once above SQL would run, you'll see a result similar to the one attached below.

Output				
Action Output				
#	Time	Action	Message	
✓ 1	21:51:34	CREATE DATABASE ORG	1 row(s) affected	
✓ 2	21:51:34	SHOW DATABASES	7 row(s) returned	
✓ 3	21:51:34	USE ORG	0 row(s) affected	
✓ 4	21:51:34	CREATE TABLE Worker (WORKER_ID INT NOT NULL PRIMARY KEY AUT...	0 row(s) affected	
✓ 5	21:51:34	INSERT INTO Worker (WORKER_ID, FIRST_NAME, LAST_NAME, SALARY...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	
✓ 6	21:51:34	CREATE TABLE Bonus (WORKER_REF_ID INT, BONUS_AMOUNT INT(10),...	0 row(s) affected	
✓ 7	21:51:34	INSERT INTO Bonus (WORKER_REF_ID, BONUS_AMOUNT, BONUS_DAT...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	

Creating Sample Data to Practice SQL Skill.

Q-1. Write An SQL Query To Fetch “FIRST_NAME” From Worker Table Using The Alias Name As <WORKER_NAME>.

Ans.

The required query is:

```
Select FIRST_NAME AS WORKER_NAME from Worker;
```

Q-2. Write An SQL Query To Fetch “FIRST_NAME” From Worker Table In Upper Case.

Ans.

The required query is:

```
Select upper(FIRST_NAME) from Worker;
```

Q-3. Write An SQL Query To Fetch Unique Values Of DEPARTMENT From Worker Table.

Ans.

The required query is:

```
Select distinct DEPARTMENT from Worker;
```

Q-4. Write An SQL Query To Print First Three Characters Of FIRST_NAME From Worker Table.

Ans.

The required query is:

```
Select substring(FIRST_NAME,1,3) from Worker;
```

Q-5. Write An SQL Query To Find The Position Of The Alphabet ('A') In The First Name Column 'Amitabh' From Worker Table.

Ans.

The required query is:

```
Select INSTR(FIRST_NAME, BINARY'a') from Worker where FIRST_NAME = 'Amitabh';
```

Notes.

- The INSTR method is in case-sensitive by default.
- Using Binary operator will make INSTR work as the case-sensitive function.

Q-6. Write An SQL Query To Print The FIRST_NAME From Worker Table After Removing White Spaces From The Right Side.

Ans.

The required query is:

```
Select RTRIM(FIRST_NAME) from Worker;
```

Q-7. Write An SQL Query To Print The DEPARTMENT From Worker Table After Removing White Spaces From The Left Side.

Ans.

The required query is:

```
Select LTRIM(DEPARTMENT) from Worker;
```

Q-8. Write An SQL Query That Fetches The Unique Values Of DEPARTMENT From Worker Table And Prints Its Length.

Ans.

The required query is:

```
Select distinct length(DEPARTMENT) from Worker;
```

Q-9. Write An SQL Query To Print The FIRST_NAME From Worker Table After Replacing 'A' With 'A'.

Ans.

The required query is:

```
Select REPLACE(FIRST_NAME,'a','A') from Worker;
```

Q-10. Write An SQL Query To Print The FIRST_NAME And LAST_NAME From Worker Table Into A Single Column COMPLETE_NAME. A Space Char Should Separate Them.

Ans.

The required query is:

```
Select CONCAT(FIRST_NAME, ' ', LAST_NAME) AS 'COMPLETE_NAME' from Worker;
```

Q-11. Write An SQL Query To Print All Worker Details From The Worker Table Order By FIRST_NAME Ascending.

Ans.

The required query is:

```
Select * from Worker order by FIRST_NAME asc;
```

Q-12. Write An SQL Query To Print All Worker Details From The Worker Table Order By FIRST_NAME Ascending And DEPARTMENT Descending.

Ans.

The required query is:

```
Select * from Worker order by FIRST_NAME asc,DEPARTMENT desc;
```

Q-13. Write An SQL Query To Print Details For Workers With The First Name As “Vipul” And “Satish” From Worker Table.

Ans.

The required query is:

```
Select * from Worker where FIRST_NAME in ('Vipul','Satish');
```

Q-14. Write An SQL Query To Print Details Of Workers Excluding First Names, “Vipul” And “Satish” From Worker Table.

Ans.

The required query is:

```
Select * from Worker where FIRST_NAME not in ('Vipul','Satish');
```

Q-15. Write An SQL Query To Print Details Of Workers With DEPARTMENT Name As “Admin”.

Ans.

The required query is:

```
Select * from Worker where DEPARTMENT like 'Admin%';
```

Q-16. Write An SQL Query To Print Details Of The Workers Whose FIRST_NAME Contains ‘A’.

Ans.

The required query is:

```
Select * from Worker where FIRST_NAME like '%a%';
```

Q-17. Write An SQL Query To Print Details Of The Workers Whose FIRST_NAME Ends With 'A'.

Ans.

The required query is:

```
Select * from Worker where FIRST_NAME like '%a';
```

Q-18. Write An SQL Query To Print Details Of The Workers Whose FIRST_NAME Ends With 'H' And Contains Six Alphabets.

Ans.

The required query is:

```
Select * from Worker where FIRST_NAME like '_____h';
```

Q-19. Write An SQL Query To Print Details Of The Workers Whose SALARY Lies Between 100000 And 500000.

Ans.

The required query is:

```
Select * from Worker where SALARY between 100000 and 500000;
```

Q-20. Write An SQL Query To Print Details Of The Workers Who Have Joined In Feb'2014.

Ans.

The required query is:

```
Select * from Worker where year(JOINING_DATE) = 2014 and month(JOINING_DATE) = 2;
```

Q-21. Write An SQL Query To Fetch The Count Of Employees Working In The Department 'Admin'.

Ans.

The required query is:

```
SELECT COUNT(*) FROM worker WHERE DEPARTMENT = 'Admin';
```

Q-22. Write An SQL Query To Fetch Worker Names With Salaries >= 50000 And <= 100000.

Ans.

The required query is:

```
SELECT CONCAT(FIRST_NAME, ' ', LAST_NAME) As Worker_Name, Salary
FROM worker
WHERE WORKER_ID IN
(SELECT WORKER_ID FROM worker
WHERE Salary BETWEEN 50000 AND 100000);
```

Q-23. Write An SQL Query To Fetch The No. Of Workers For Each Department In The Descending Order.

Ans.

The required query is:

```
SELECT DEPARTMENT, count(WORKER_ID) No_Of_Workers
FROM worker
GROUP BY DEPARTMENT
ORDER BY No_Of_Workers DESC;
```

Q-24. Write An SQL Query To Print Details Of The Workers Who Are Also Managers.

Ans.

The required query is:

```
SELECT DISTINCT W.FIRST_NAME, T.WORKER_TITLE
FROM Worker W
INNER JOIN Title T
ON W.WORKER_ID = T.WORKER_REF_ID
AND T.WORKER_TITLE in ('Manager');
```

Q-25. Write An SQL Query To Fetch Duplicate Records Having Matching Data In Some Fields Of A Table.

Ans.

The required query is:

```
SELECT WORKER_TITLE, AFFECTED_FROM, COUNT(*)  
FROM Title  
GROUP BY WORKER_TITLE, AFFECTED_FROM  
HAVING COUNT(*) > 1;
```

Q-26. Write An SQL Query To Show Only Odd Rows From A Table.

Ans.

The required query is:

```
SELECT * FROM Worker WHERE MOD (WORKER_ID, 2) <> 0;
```

Q-27. Write An SQL Query To Show Only Even Rows From A Table.

Ans.

The required query is:

```
SELECT * FROM Worker WHERE MOD (WORKER_ID, 2) = 0;
```

Q-28. Write An SQL Query To Clone A New Table From Another Table.

Ans.

The general query to clone a table with data is:

```
SELECT * INTO WorkerClone FROM Worker;
```

The general way to clone a table without information is:

```
SELECT * INTO WorkerClone FROM Worker WHERE 1 = 0;
```

An alternate way to clone a table (for MySQL) without is:

```
CREATE TABLE WorkerClone LIKE Worker;
```

Q-29. Write An SQL Query To Fetch Intersecting Records Of Two Tables.

Ans.

The required query is:

```
(SELECT * FROM Worker)  
INTERSECT  
(SELECT * FROM WorkerClone);
```

Q-30. Write An SQL Query To Show Records From One Table That Another Table Does Not Have.

Ans.

The required query is:

```
SELECT * FROM Worker  
MINUS  
SELECT * FROM Title;
```

Q-31. Write An SQL Query To Show The Current Date And Time.

Ans.

Following MySQL query returns the current date:

```
SELECT CURDATE();
```

Following MySQL query returns the current date and time:

```
SELECT NOW();
```

Following SQL Server query returns the current date and time:

```
SELECT getdate();
```

Following Oracle query returns the current date and time:

```
SELECT SYSDATE FROM DUAL;
```

Q-32. Write An SQL Query To Show The Top N (Say 10) Records Of A Table.

Ans.

Following MySQL query will return the top n records using the LIMIT method:

```
SELECT * FROM Worker ORDER BY Salary DESC LIMIT 10;
```

Following SQL Server query will return the top n records using the TOP command:

```
SELECT TOP 10 * FROM Worker ORDER BY Salary DESC;
```

Following Oracle query will return the top n records with the help of ROWNUM:

```
SELECT * FROM (SELECT * FROM Worker ORDER BY Salary DESC)
WHERE ROWNUM <= 10;
```

Q-33. Write An SQL Query To Determine The Nth (Say N=5) Highest Salary From A Table.

Ans.

The following MySQL query returns the nth highest salary:

```
SELECT Salary FROM Worker ORDER BY Salary DESC LIMIT n-1,1;
```

The following SQL Server query returns the nth highest salary:

```
SELECT TOP 1 Salary
FROM (
  SELECT DISTINCT TOP n Salary
  FROM Worker
  ORDER BY Salary DESC
)
ORDER BY Salary ASC;
```

Q-34. Write An SQL Query To Determine The 5th Highest Salary Without Using TOP Or Limit Method.

Ans.

The following query is using the correlated subquery to return the 5th highest salary:

```
SELECT Salary
FROM Worker W1
WHERE 4 = (
  SELECT COUNT( DISTINCT ( W2.Salary ) )
  FROM Worker W2
  WHERE W2.Salary >= W1.Salary
);
```

Use the following generic method to find nth highest salary without using TOP or limit.

```
SELECT Salary
FROM Worker W1
WHERE n-1 = (
  SELECT COUNT( DISTINCT ( W2.Salary ) )
  FROM Worker W2
```

```
WHERE W2.Salary >= W1.Salary  
);
```

Q-35. Write An SQL Query To Fetch The List Of Employees With The Same Salary.

Ans.

The required query is:

```
Select distinct W.WORKER_ID, W.FIRST_NAME, W.Salary  
from Worker W, Worker W1  
where W.Salary = W1.Salary  
and W.WORKER_ID != W1.WORKER_ID;
```

Q-36. Write An SQL Query To Show The Second Highest Salary From A Table.

Ans.

The required query is:

```
Select max(Salary) from Worker  
where Salary not in (Select max(Salary) from Worker);
```

Q-37. Write An SQL Query To Show One Row Twice In Results From A Table.

Ans.

The required query is:

```
select FIRST_NAME, DEPARTMENT from worker W where W.DEPARTMENT='HR'  
union all  
select FIRST_NAME, DEPARTMENT from Worker W1 where W1.DEPARTMENT='HR';
```

Q-38. Write An SQL Query To Fetch Intersecting Records Of Two Tables.

Ans.

The required query is:

```
(SELECT * FROM Worker)  
INTERSECT  
(SELECT * FROM WorkerClone);
```

Q-39. Write An SQL Query To Fetch The First 50% Records From A Table.

Ans.

The required query is:

```
SELECT *  
FROM WORKER  
WHERE WORKER_ID <= (SELECT count(WORKER_ID)/2 from Worker);
```

Q-40. Write An SQL Query To Fetch The Departments That Have Less Than Five People In It.

Ans.

The required query is:

```
SELECT DEPARTMENT, COUNT(WORKER_ID) as 'Number of Workers' FROM Worker GROUP  
BY DEPARTMENT HAVING COUNT(WORKER_ID) < 5;
```

Q-41. Write An SQL Query To Show All Departments Along With The Number Of People In There.

Ans.

The following query returns the expected result:

```
SELECT DEPARTMENT, COUNT(DEPARTMENT) as 'Number of Workers' FROM Worker GROUP  
BY DEPARTMENT;
```

Q-42. Write An SQL Query To Show The Last Record From A Table.

Ans.

The following query will return the last record from the Worker table:

```
Select * from Worker where WORKER_ID = (SELECT max(WORKER_ID) from Worker);
```

Q-43. Write An SQL Query To Fetch The First Row Of A Table.

Ans.

The required query is:

```
Select * from Worker where WORKER_ID = (SELECT min(WORKER_ID) from Worker);
```

Q-44. Write An SQL Query To Fetch The Last Five Records From A Table.

Ans.

The required query is:

```
SELECT * FROM Worker WHERE WORKER_ID <=5
UNION
SELECT * FROM (SELECT * FROM Worker W order by W.WORKER_ID DESC) AS W1 WHERE
W1.WORKER_ID <=5;
```

Q-45. Write An SQL Query To Print The Name Of Employees Having The Highest Salary In Each Department.

Ans.

The required query is:

```
SELECT t.DEPARTMENT,t.FIRST_NAME,t.Salary from(SELECT max(Salary) as
TotalSalary,DEPARTMENT from Worker group by DEPARTMENT) as TempNew
Inner Join Worker t on TempNew.DEPARTMENT=t.DEPARTMENT
and TempNew.TotalSalary=t.Salary;
```

Q-46. Write An SQL Query To Fetch Three Max Salaries From A Table.

Ans.

The required query is:

```
SELECT distinct Salary from worker a WHERE 3 >= (SELECT count(distinct Salary) from worker
b WHERE a.Salary <= b.Salary) order by a.Salary desc;
```

Q-47. Write An SQL Query To Fetch Three Min Salaries From A Table.

Ans.

The required query is:

```
SELECT distinct Salary from worker a WHERE 3 >= (SELECT count(distinct Salary) from worker
b WHERE a.Salary >= b.Salary) order by a.Salary desc;
```

Q-48. Write An SQL Query To Fetch Nth Max Salaries From A Table.

Ans.

The required query is:

```
SELECT distinct Salary from worker a WHERE n >= (SELECT count(distinct Salary) from worker b WHERE a.Salary <= b.Salary) order by a.Salary desc;
```

Q-49. Write An SQL Query To Fetch Departments Along With The Total Salaries Paid For Each Of Them.

Ans.

The required query is:

```
SELECT DEPARTMENT, sum(Salary) from worker group by DEPARTMENT;
```

Q-50. Write An SQL Query To Fetch The Names Of Workers Who Earn The Highest Salary.

Ans.

The required query is:

```
SELECT FIRST_NAME, SALARY from Worker WHERE SALARY=(SELECT max(SALARY) from Worker);
```

What is a SQL join?

A SQL join is a Structured Query Language (**SQL**) instruction to combine data from two sets of data (i.e. two tables). Before we dive into the details of a SQL join, let's briefly discuss what SQL is, and why someone would want to perform a SQL join.

SQL is a special-purpose programming language designed for managing information in a relational database management system (**RDBMS**). The word relational here is key; it specifies that the database management system is organized in such a way that there are clear relations defined between different sets of data.

Typically, you need to extract, transform, and load data into your RDBMS before you're able to manage it using SQL, which you can accomplish by using a tool like [Stitch](#).

Relational Database Example

Imagine you're running a store and would like to record information about your customers and their orders. By using a relational database, you can save this information as two tables that represent two distinct entities: customers and orders.

Customers

customer_id	first_name	last_name	email	address	city	state	zip
1	George	Washington	gWASHINGTON@usa.gov	3200 Mt Vernon Hwy	Mount Vernon	VA	22121
2	John	Adams	jAdams@usa.gov	1250 Hancock St	Quincy	MA	02169
3	Thomas	Jefferson	tJefferson@usa.gov	931 Thomas Jefferson Pkwy	Charlottesville	VA	22902
4	James	Madison	jMadison@usa.gov	11350 Constitution Hwy	Orange	VA	22960
5	James	Monroe	jMonroe@usa.gov	2050 James Monroe Pkwy	Charlottesville	VA	22902

Here, information about each customer is stored in its own row, with columns specifying different bits of information, including their first name, last name, and email address. Additionally, we associate a unique customer number, or primary key, with each customer record.

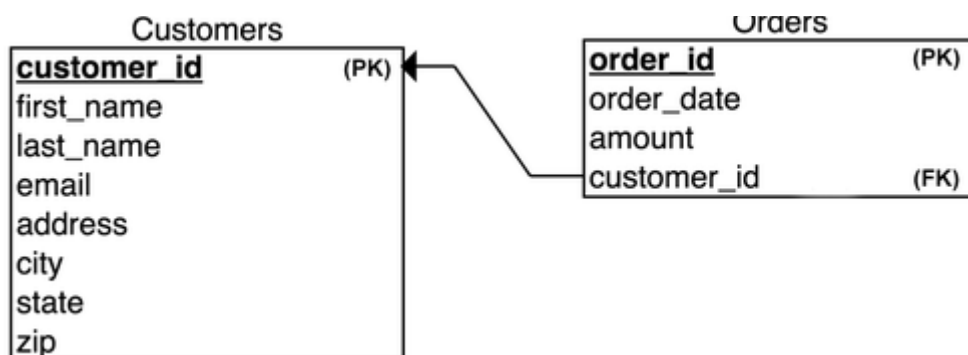
Orders

order_id	order_date	amount	customer_id
1	07/04/1776	\$234.56	1
2	03/14/1760	\$78.50	3
3	05/23/1784	\$124.00	2
4	09/03/1790	\$65.50	3
5	07/21/1795	\$25.50	10
6	11/27/1787	\$14.40	9

Again, each row contains information about a specific order. Each order has its own unique identification key – **order_id** for this table – assigned to it as well.

Relational Model

You’ve probably noticed that these two examples share similar information. You can see these simple relations diagrammed below:



Note that the orders table contains two keys: one for the order and one for the customer who placed that order. In scenarios when there are multiple keys in a table, the key that

refers to the entity being described in that table is called the **primary key** (PK) and other key is called a **foreign key** (FK).

In our example, `order_id` is a primary key in the orders table, while `customer_id` is both a primary key in the customers table and a foreign key in the orders table. Primary and foreign keys are essential to describing relations between the tables, and in performing SQL joins.

SQL Join Example

Let's say we want to find all orders placed by a particular customer. We can do this by joining the customers and orders tables together using the relationship established by the `customer_id` key:

```
select order_date, order_amount
from customers
join orders
  on customers.customer_id = orders.customer_id
where customer_id = 3
```

Here, we're joining the two tables using the `join` keyword, and specifying what key to use when joining the tables in the `on customers.customer_id = orders.customer_id` line following the join statement. Here is the result of the above SQL query, which includes two orders placed by Thomas Jefferson (`customer_id = 3`):

order_id	order_date	order_amount
2	3/14/1760	\$78.50
4	9/03/1790	\$65.50

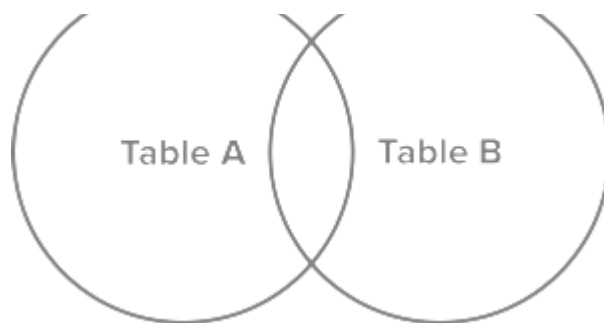
This particular join is an example of an "inner" join. Depending on the kind of analysis you'd like to perform, you may want to use a different method. There are actually a number of different ways to join the two tables together, depending on your application. The next section will explain inner, left, right, and full joins, and provide examples using the data tables used above.

Basic SQL Join Types

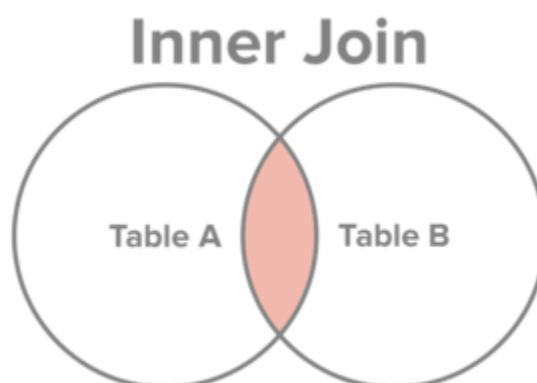
There are four basic types of SQL joins: inner, left, right, and full. The easiest and most intuitive way to explain the difference between these four types is by using a Venn diagram, which shows all possible logical relations between data sets.

Again, it's important to stress that before you can begin using any join type, you'll need to extract the data and load it into an RDBMS like Amazon Redshift, where you can query tables from multiple sources. You build that process manually, or you can use an ETL service like [Stitch](#), which automates that process for you.

Let's say we have two sets of data in our relational database: table A and table B, with some sort of relation specified by primary and foreign keys. The result of joining these tables together can be visually represented by the following diagram:

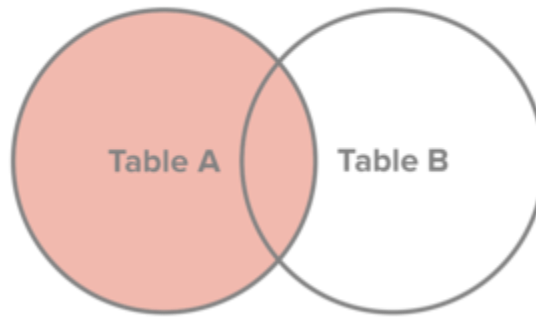


The extent of the overlap, if any, is determined by how many records in Table A match the records in Table B. Depending on what subset of data we would like to select from the two tables, the four join types can be visualized by highlighting the corresponding sections of the Venn diagram:



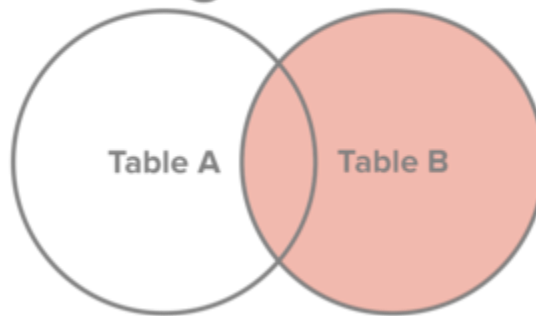
Select all records from Table A and Table B, where the join condition is met.

Left Join



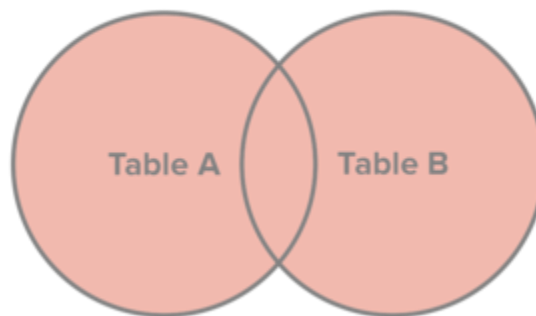
Select all records from Table A, along with records from Table B for which the join condition is met (if at all).

Right Join



Select all records from Table B, along with records from Table A for which the join condition is met (if at all).

Full Join



Select all records from Table A and Table B, regardless of whether the join condition is met or not.

Examples of SQL Join Types

Let's use the tables we introduced in the “What is a SQL join?” section to show examples of these joins in action. The relationship between the two tables is specified by the `customer_id` key, which is the "primary key" in customers table and a "foreign key" in the orders table:

customer_id	first_name	last_name	email	address	city	state	zipcode
1	George	Washington	g washington@usa.gov	3200 Mt Vernon Hwy	Mount Vernon	VA	22121
2	John	Adams	j adams@usa.gov	1250 Hancock St	Quincy	MA	02169
3	Thomas	Jefferson	t jefferson@usa.gov	931 Thomas Jefferson Pkwy	Charlottesville	VA	22902
4	James	Madison	j madison@usa.gov	11350 Constitution Hwy	Orange	VA	22960
5	James	Monroe	j monroe@usa.gov	2050 James Monroe Parkway	Charlottesville	VA	22902
order_id		order_date	amount	customer_id			
1		07/04/1776	\$234.56	1			
2		03/14/1760	\$78.50	3			
3		05/23/1784	\$124.00	2			
4		09/03/1790	\$65.50	3			
5		07/21/1795	\$25.50	10			
6		11/27/1787	\$14.40	9			

Note that (1) not every customer in our customers table has placed an order and (2) there are a few orders for which no customer record exists in our customers table.

Inner Join

Let's say we wanted to get a list of those customers who placed an order and the details of the order they placed. This would be a perfect fit for an inner join, since an inner join returns records at the intersection of the two tables.

```
select first_name, last_name, order_date, order_amount
from customers c
inner join orders o
on c.customer_id = o.customer_id
```

first_name	last_name	order_date	order_amount
George	Washington	07/4/1776	\$234.56
John	Adams	05/23/1784	\$124.00
Thomas	Jefferson	03/14/1760	\$78.50
Thomas	Jefferson	09/03/1790	\$65.50

Note that only George Washington, John Adams and Thomas Jefferson placed orders, with Thomas Jefferson placing two separate orders on 3/14/1760 and 9/03/1790.

Left Join

If we wanted to simply append information about orders to our customers table, regardless of whether a customer placed an order or not, we would use a left join. A left join returns all records from table A and any matching records from table B.

```
select first_name, last_name, order_date, order_amount
from customers c
left join orders o
on c.customer_id = o.customer_id
```

first_name	last_name	order_date	order_amount
George	Washington	07/04/1776	\$234.56
John	Adams	05/23/1784	\$124.00
Thomas	Jefferson	03/14/1760	\$78.50

Thomas	Jefferson	09/03/1790	\$65.50
James	Madison	NULL	NULL
James	Monroe	NULL	NULL

Note that since there were no matching records for James Madison and James Monroe in our orders table, the `order_date` and `order_amount` are `NULL`, which simply means there is no data for these fields.

So why would this be useful? By simply adding a “where `order_date` is `NULL`” line to our SQL query, it returns a list of all customers who have not placed an order:

```
select first_name, last_name, order_date, order_amount
from customers c
left join orders o
on c.customer_id = o.customer_id
where order_date is NULL
```

Right Join

Right join is a mirror version of the left join and allows to get a list of all orders, appended with customer information.

```
select first_name, last_name, order_date, order_amount
from customers c
right join orders o
on c.customer_id = o.customer_id
```

first_name	last_name	order_date	order_amount
George	Washington	07/04/1776	\$234.56
Thomas	Jefferson	03/14/1760	\$78.50
John	Adams	05/23/1784	\$124.00
Thomas	Jefferson	09/03/1790	\$65.50
NULL	NULL	07/21/1795	\$25.50
NULL	NULL	11/27/1787	\$14.40

Note that since there were no matching customer records for orders placed in 1795 and 1787, the `first_name` and `last_name` fields are `NULL` in the resulting set.

Also note that the order in which the tables are joined is important. We are right joining the orders table to the customers table. If we were to right join the customers table to the orders table, the result would be the same as left joining the orders table to the customers table.

Why is this useful? Simply adding a “where first_name is NULL” line to our SQL query returns a list of all orders for which we failed to record information about the customers who placed them:

```
select first_name, last_name, order_date, order_amount
from customers c
right join orders o
on c.customer_id = o.customer_id
where first_name is NULL
```

Full Join

Finally, for a list of all records from both tables, we can use a full join.

```
select first_name, last_name, order_date, order_amount
from customers c
full join orders o
on c.customer_id = o.customer_id
```

first_name	last_name	order_date	order_amount
George	Washington	07/04/1776	\$234.56
Thomas	Jefferson	03/14/1760	\$78.50
John	Adams	05/23/1784	\$124.00
Thomas	Jefferson	09/03/1790	\$65.50
NULL	NULL	07/21/1795	\$25.50
NULL	NULL	11/27/1787	\$14.40
James	Madison	NULL	NULL
James	Monroe	NULL	NULL

Site for joins :- <http://www.sql-join.com/sql-join-types>

